the per-step label A_i, and the set of available tools defines the answer space for these datasets.

3.7. Models

We consider the Qwen3 family of language reasoning models. From Qwen3 we use the dense variants Qwen3-4B, Qwen3-8B and the MoE variant Qwen3-30B-A3B (Yang et al., 2025). We use the query encoder from (Reimers & Gurevych, 2019) for comparing search queries. For all models we use the recommended generation settings unless stated otherwise. Specifically, we set temperature $= 0.6$ for sampling during both full generation and forced answer completion.

4. Results

We report results on how the model's predicted answer evolves across reasoning steps using forced answer completion, and how these dynamics can be used to reduce the number of generated reasoning tokens. We analyze the frequency and distribution of answer switches and transient answer flips across tasks and models, and measure how many reasoning tokens are generated after the final answer switch. Building on these observations, we evaluate early stopping strategies that halt generation at intermediate reasoning steps and quantify the resulting trade-off between token usage and final answer accuracy.

4.1. Properties of Answer Trajectories

We discuss observed properties of answer trajectories obtained via forced answer completion, focusing on how predicted answers change across reasoning steps and how these changes are distributed across models and tasks.

4.1.1. Transient Answer Flips

We first examine the occurrence of transient answer flips, defined as short-lived deviations in the predicted answer that revert within a small number of reasoning steps. Across all evaluated models and task settings, transient answer flips are common, indicating frequent, brief changes in the predicted answer that do not persist. Table 1 reports the average number of transient answer flips per example for each model across tasks. We observe that transient flips occur frequently, $\approx$ **3 TAFs per reasoning trace**, across all models for multiple-choice and numeric answers. We find a lower transient answer flip count for tool-use and search which is consistent with the lower answer switches.

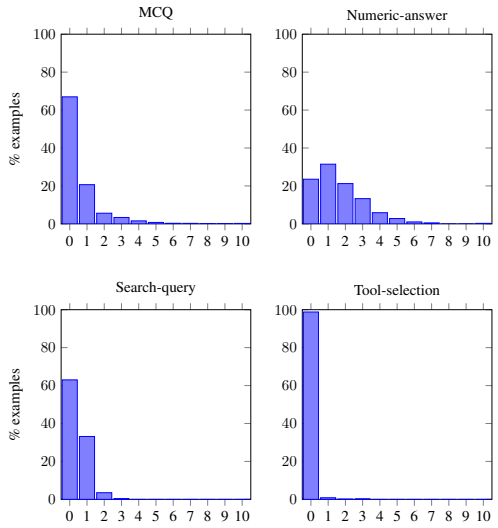


Figure 2. Distribution of answer switch counts across denoised answer trajectories for different task types using Qwen3-4B.

4.1.2. Answer Trajectories with No Answer Switches

Next, we analyze the fraction of examples in which the predicted answer does not change at any reasoning step, i.e., the answer trajectory satisfies $A_0 = A_1 = \cdots = A_n$ or $t^\star = -1$. In such cases, the predicted answer under forced completion remains identical to the non-reasoning baseline throughout the entire reasoning trace.

Figure 2 shows the distribution of the number of answer switches across examples. A substantial fraction of examples (**$\approx$ 50% examples in the MCQ dataset across all models**) exhibit zero answer switches, indicating that for these inputs the predicted answer is unchanged by the generated reasoning steps.

4.1.3. Limited Number of Answer Switches When Switches Occur

We consider examples in which the predicted answer does change at least once. Among these examples, the total number of answer switches is typically small relative to the length of the reasoning trace.

Figure 2 presents the distribution of answer switches. The majority of such examples exhibit only a small number of switches, with long stretches of reasoning steps between successive changes.

4.1.4. Tool-Selection Tasks Exhibit Near-Zero Answer Switches

For tool-selection datasets, from Table 1 we observe that the predicted answer is highly stable across reasoning steps as seen from ≈ 0 Answer Switches for the Tool Selection Dataset across all models. In the vast majority of exam-

Table 1. Answer-trajectory Metrics computed across different datasets. Except for Answer Switches, lower values of all metrics indicate fewer redundant tokens generated after the final decision, and therefore greater efficiency. For $\Pr(t^\star = -1)$, we compute the fraction of examples with no answer switch on the raw traces and on the hold-for-k denoised traces, respectively. For T_{after} each entry shows "tokens (percentage of reasoning tokens)". We present detailed distributions of these metrics in Appendix D

.

Dataset	Model	$\Pr(t^\star = -1)$ Raw% / Denoised%	AnswerSwitches Raw / Denoised	TAF(T)	T_{after} Raw / Denoised
MCQ	Qwen3-4B	58.6 / 67.0	3.59 / 0.61	2.02	555.2 (35.7%) / 867.1 (43.1%)
	Qwen3-8B	50.5 / 59.2	4.30 / 0.80	2.29	663.0 (35.9%) / 1033.4 (43.4%)
	Qwen3-14B	55.4 / 65.3	3.11 / 0.64	1.64	609.9 (35.4%) / 783.5 (39.3%)
	Qwen3-30B-A3B	67.9 / 73.1	2.17 / 0.44	1.16	641.5 (40.1%) / 868.5(45.5%)
	GPT-OSS-20B (High)	56.1 / 63.6	2.16 / 0.63	0.97	583.7 (47.0%) / 712.6 (46.3%)
	GPT-OSS-20B (Medium)	50.3 / 60.8	2.91 / 0.75	1.36	284.1 (35.7%) / 361.2 (31.2%)
Numeric-answer	Qwen3-4B	19.4 / 23.5	7.15 / 1.64	3.40	846.7 (42.6%) / 1192.9 (51.8%)
	Qwen3-8B	20.8 / 24.8	9.26 / 1.74	4.68	1280 (45.8%) / 1904.4 (59.9%)
	Qwen3-14B	18.2 / 27.3	8.77 / 1.80	4.43	1271.7 (50.5%) / 1717.3 (62.4%)
	Qwen3-30B-A3B	31.2 / 37.5	6.75 / 1.06	3.75	1203.8 (50.5%) / 1830.4 (70.2%)
	GPT-OSS-20B (High)	9.9 / 14.6	13.02 / 2.35	5.82	823.6 (39.5%) / 1437.5 (51.9%)
	GPT-OSS-20B (Medium)	8.0 / 13.3	13.83 / 2.72	5.77	307.0 (25.4%) / 466.8 (27.7%)
Search-query	Qwen3-4B	53.5 / 63.0	0.85 / 0.41	0.26	148.1 (64.7%) / 106.5 (44.7%)
	Qwen3-8B	60.1 / 74.5	0.87 / 0.28	0.38	145.5 (58.6%) / 121.6 (46.6%)
	Qwen3-14B	68.5 / 79.8	0.61 / 0.22	0.26	114.4 (54.4%) / 78.5 (36.3%)
	Qwen3-30B-A3B	60.1 / 80.2	0.91 / 0.22	0.40	95.8 (45.2%) / 82.2 (35.3%)
	GPT-OSS-20B (High)	74.7 / 97.3	0.55 / 0.03	0.29	36.8 (19.4%) / 63.4 (29.4%)
	GPT-OSS-20B (Medium)	88.4 / 99.6	0.17 / 0.00	0.07	9.1 (15.5%) / 12.4 (10.1%)
Tool-selection	Qwen3-4B	97.7 / 98.8	0.21 / 0.02	0.14	647.0 (41.3%) / 855.7 (56.6%)
	Qwen3-8B	97.4 / 98.6	0.11 / 0.02	0.06	338 (44.9%) / 465.3 (46.8%)
	Qwen3-14B	96.4 / 98.0	0.16 / 0.03	0.09	331.6 (44.6%) / 306.6 (38.5%)
	Qwen3-30B-A3B	99.0 / 100	0.02 / 0.00	0.01	37.0 (23.6%) / 37.0 (23.6%)
	GPT-OSS-20B (High)	87.3 / 96.1	0.79 / 0.08	0.44	576.9 (53.1%) / 677.8 (54.8%)
	GPT-OSS-20B (Medium)	93.0 / 97.1	0.31 / 0.05	0.17	169.5 (49.6%) / 147.2 (39.1%)

ples, the model selects the same tool from the initial forced completion onward, resulting in almost no answer switches throughout the reasoning trace. Consequently, early stopping at intermediate steps does not alter the selected tool in practice.

Because the predicted tool remains unchanged, early stopping induces no measurable degradation in task performance. For this reason, we do not report accuracy–token trade-off curves for tool-selection tasks, as the accuracy is effectively invariant to the stopping point under our evaluation protocol.

Despite this stability, we include hidden states from tool-selection examples when training the generic probe. Although these examples provide limited signal for detecting answer switches, they expose the probe to reasoning representations from a distinct task regime, improving robustness and cross-task generalization. In particular, incorporating tool-selection data helps the generic probe learn representations associated with early decision commitment.

4.2. Tokens After Final Answer Switch

We now analyze how many reasoning tokens are generated after the model's final answer switch, as defined in Section 3.3.1. For each example, we measure the number of tokens generated after the last reasoning step at which the predicted answer changes.

Table 1 shows the average number of tokens generated after the final answer switch per example. We observe that a large fraction of the total reasoning tokens **(≈50% of reasoning tokens across all dataset and models)** are generated after the predicted answer has already reached its final value.

These tokens represent reasoning that does not alter the model's predicted answer under forced completion. As a result, the measured quantity provides an upper bound on the number of tokens that could be avoided by an ideal early stopping strategy that halts generation immediately after the final answer switch.

While practical early stopping methods may not perfectly identify this point, the observed distributions indicate that a substantial portion of generated reasoning tokens occur after the final answer switch, leaving significant room for reducing token usage without changing the predicted answer.

4.3. Evaluation on Harder Reasoning Benchmarks

We extend our analysis to substantially harder reasoning benchmarks to evaluate whether the observed answer trajectory dynamics persist in more challenging settings. Specifically, we consider Humanity's Last Exam (Center for AI Safety et al., 2026), GPQA-Diamond (Rein et al., 2023), and AIME 2026 problems (Balunović et al., 2025), which are designed to require deeper multi-step reasoning and exhibit

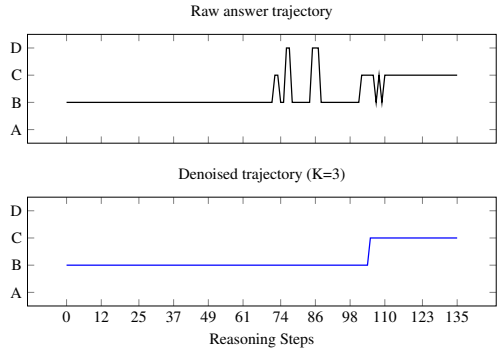


Figure 3. Illustration of Answer Switch Denoising for MCQ task. After denoising, the decision path becomes clearer across the entire reasoning process.

lower overall accuracy compared to the datasets considered earlier.

Across all three benchmarks, we observe qualitatively consistent behavior with our main results. In particular, the predicted answer often stabilizes before the completion of the full reasoning trace, even in these more demanding settings. Although these datasets exhibit higher answer-switch counts and require longer reasoning chains, the final answer switch typically occurs well before the end of generation.

We measure the number of reasoning tokens generated after the final answer switch as defined in Section 3.5. Despite the increased task difficulty, a substantial fraction of reasoning tokens are generated after the predicted answer has already stabilized. On average, we find that approximately 32%–67% of the total reasoning tokens are produced after the final answer switch across these datasets.

These results indicate that the temporal gap between decision formation and subsequent reasoning is not limited to moderate-difficulty or proxy tasks, but persists even in substantially harder reasoning regimes. This further supports the generality of the early decision-making phenomenon observed in our study. We list the metrics calculated in Appendix F

4.4. Effect of Denoising

We evaluate the effect of answer trajectory denoising on the structure of predicted answer sequences. The denoising procedure is applied to suppress transient answer flips while preserving sustained answer changes.

Figure 3 compares the original and denoised answer trajectory of an example: Short-lived answer changes present in the raw trajectory are removed after denoising, while longer-lasting answer changes are retained.

We also show the effect of denoising at the dataset level in by the distribution plots in Appendix D. Compared to the original distribution, the denoised distribution exhibits fewer examples with a large number of answer switches, and the mass is concentrated on smaller switch counts. For example, as seen in Table 1, the average answer switches decrease when the trajectory is denoised, $\approx \mathbf{8} \rightarrow \approx \mathbf{2}$ **for the Numeric-answer dataset across all models**.

These results show that denoising removes transient answer flips and yields answer switch distributions that are less spread out, while preserving the overall structure of sustained answer changes.

5. Early Stopping and Performance Trade-Offs

We evaluate two early-stopping strategies for determining when to terminate a model's reasoning process, i.e., when intermediate predictions are redundant to the final decision.

5.1. Early Stopping Gates

We introduce early-stopping gates that operate over partial reasoning prefixes and produce a scalar confidence score indicating whether generation can be terminated at a given step. These gates do not alter the model's generation process; rather, they observe intermediate states and define stopping rules based on predefined criteria.

Formally, at each reasoning step $i \in \{0, \ldots, n\}$, a gate observes information available up to that step and outputs a scalar score:

$$m_i = \mathcal{M}(C, R_1, \ldots, R_i), \qquad m_i \in [0, 1], \tag{6}$$

where $\mathcal{M}$ denotes the gate function. The score m_i is interpreted as the gate's confidence that further reasoning is unnecessary.

Given a threshold $\tau \in [0, 1]$, the corresponding stopping rule halts generation at the earliest step

$$s = \min\{\, i \mid m_i \geq \tau \,\}. \tag{7}$$

Varying τ allows explicit control over the tradeoff between final answer accuracy and token savings.

We consider the two gating criteria, random gating and Probe-Based gate. The **random gate** outputs a confidence score drawn independently at each reasoning step. Specifically, at step i the gate produces $m_i \sim \mathrm{Uniform}(0, 1)$.

The **probe-based gate** uses a learned linear transformation over a layer's intermediate model representations to estimate whether further reasoning is necessary. Let $h_i^l \in \mathbb{R}^d$ denote the final token hidden state extracted from the model's lth layer at reasoning step i. The gate computes

$$m_i = \sigma(w^\top h_i^l + b), \tag{8}$$

where $w \in \mathbb{R}^d$, $b \in \mathbb{R}$ are learned parameters and $\sigma(\cdot)$ denotes the sigmoid function, ensuring $m_i \in (0, 1)$. The gate is trained to predict whether the predicted answer at step i matches the eventual final answer produced at step n. The supervision signal is derived post hoc from full reasoning traces and does not influence the generation process.

5.2. Experimental Setup

Reasoning trace collection. For each dataset D, we first collect full reasoning traces by running the model to completion. For each example, this yields a reasoning trace $R = (R_1, \ldots, R_n)$. We then use forced answer completion to get an answer trajectory $T = (A_0, \ldots, A_n)$, and the index of the final answer switch $t^\star$ as defined in Section 3.3.1. Collected traces are split at the example level into training, validation, and test sets. All probe parameters are learned using only the training split, with hyperparameters selected on the validation split. The test split is held out for final evaluation only.

Task-specific and generic probe-based gate. We train probe-based gates under two settings. In the task-specific setting, we train a separate probe for each dataset using only that dataset's training split, allowing the probe to specialize to dataset-specific reasoning dynamics. In addition, we train a generic probe on the union of training splits from all datasets. The motivation is to test whether there exists a shared, task-agnostic structure in the model's intermediate representations that reliably signals when further reasoning is unnecessary. The generic probe is evaluated on each dataset's test split without any dataset-specific adaptation. Comparing task-specific and generic probes, therefore, we examine whether early stopping can be governed by a universal signal, rather than task or dataset-specific heuristics. Further training details are given in Appendix E.

5.3. Early Stopping Setup

We plot the number of reasoning tokens saved and the resulting drop in final answer accuracy for MCQ and Numeric test sets in the left two of Figure 4, and the cosine similarity of the intermediate query and the final query for the Search test set in the right one.

First, probe-based gates in both task-specific and generic settings can effectively **save 500 tokens with less than 2% accuracy drop** on Qwen3-4B. At higher savings of 1,000 tokens, generic probing on the numeric dataset still limits the accuracy drop to roughly 4%. Although numeric tasks are inherently more sensitive to truncated reasoning, probing continues to provide a markedly better trade-off, suggesting that answer determination often precedes the completion of full arithmetic derivations.

For the search task, we measure degradation using final query cosine similarity rather than accuracy. Generic probing preserves a cosine similarity of ≈ 0.98 while saving up to 100 tokens, and remains above **0.95 even when saving 200 tokens**. In contrast, random stopping drops below 0.93 at similar token savings. This demonstrates that probing-based early stopping preserves the semantic content of the final query even when a substantial fraction of reasoning tokens is removed.

Overall, these quantitative results align with our trajectory analysis, showing that, on Qwen3-4B predicted answers change in only 32% of queries and that models generate an average of 760 additional tokens after the final answer is determined, averaged across all tasks. Probing-based early stopping capitalizes on this redundancy, **saving up to 500–900 tokens per query with only 1–5% accuracy degradation** in many regimes. While we do not observe a consistent trend with respect to model size, similar patterns emerge across different scales. Collectively, this confirms that a significant portion of chain-of-thought generation is unnecessary for correctness and can be safely eliminated using simple, inference-time heuristics. Exhaustive results are presented in Section C in the Appendix.

Individual vs Generic Probe-based gate Across tasks, generic probe closely matches token–performance trade-off achieved by individually trained task-specific gates, with no consistent loss in token savings or final answer quality.

Despite being trained without any dataset-specific supervision, the generic probe gate yields early stopping frontiers that largely overlap with those of the individual probes in Figure 4. This indicates that the intermediate representations used by the probe to decide when further reasoning is unnecessary capture signals that are largely shared across tasks. The comparable performance of the generic and individual gates therefore suggests the presence of a larger universal structure governing answer switches and early stopping, rather than task-specific stopping behavior.

6. Discussion and Limitations

Our results show that, across models and tasks, our predicted answer often stabilizes early in the reasoning process. Many examples show no answer switches at all, and when the switches do occur, their number is small. Importantly, a large portion of reasoning tokens are generated after the final answer switch, meaning they do not change the predicted answer under forced completion. This explains why early stopping can save a substantial number of tokens with only a small drop in performance. Probe-based stopping is especially effective, and the strong performance of the generic probe suggests that models encode shared signals that indicate when further reasoning is unlikely to change the answer.